

## CAMARA Y ALMACENAMIENTO

Partiendo del proyecto anterior se va a implementar una Activity que saca fotos y las almacena en el teléfono. Para ello hay que crear CameraActivity:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    tools:context=".CameraActivity">

    <Button
        android:id="@+id/btnSacarFoto"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Sacar foto" />

    <ImageView
        android:id="@+id/ivFoto"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:scaleType="fitCenter" />

    <GridView
        android:id="@+id/gvMiniaturas"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:numColumns="3" />

</LinearLayout>
```

Se programa en CentralActivity el botón que abre la cámara:

```
package com.example.ejemplodialog;

import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
```

```
import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

public class CentralActivity extends AppCompatActivity
{
    private Button btnSensores;
    private Button btnCamara;

    @Override
    protected void onCreate(Bundle savedInstanceState)
    {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable(this);
        setContentView(R.layout.activity_central);
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main),
        (v, insets) ->
        {
            Insets systemBars =
            insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top,
            systemBars.right, systemBars.bottom);
            return insets;
        });

        btnSensores = findViewById(R.id.btnSensores);
        btnSensores.setOnClickListener(new View.OnClickListener()
        {
            @Override
            public void onClick(View v)
```

```

        {
            startActivity(new Intent(getApplicationContext(),
SensoresActivity.class));
        }
    });

    btnCamara = findViewById(R.id.btncamara);
    btnCamara.setOnClickListener(new View.OnClickListener()
    {
        @Override
        public void onClick(View v)
        {
            startActivity(new Intent(getApplicationContext(),
CameraActivity.class));
        }
    });
}
}

```

Se agrega Glide a build.gradle.kts (Module :app), dentro de dependencies { ... }:

```
implementation("com.github.bumptech.glide:glide:4.16.0")
```

Para configurar el FileProvider y poder guardar las imágenes habrá que crear el archivo res/xml/file\_paths.xml:

```

<?xml version="1.0" encoding="utf-8"?>
<paths>
    <external-files-path name="fotos" path="Pictures/" />
</paths>

```

En el `AndroidManifest.xml`, dentro de `<application>`, añade el provider:

```
<provider
    android:name="androidx.core.content.FileProvider"
    android:authorities="${applicationId}.fileprovider"
    android:exported="false"
    android:grantUriPermissions="true">
    <meta-data
        android:name="android.support.FILE_PROVIDER_PATHS"
        android:resource="@xml/file_paths" />
</provider>
```

Se programa `CameraActivity.java`:

```
package com.example.ejemplodialog;

import android.net.Uri;
import android.os.Bundle;
import android.os.Environment;
import android.view.View;
import android.widget.Button;
import android.widget.ImageView;

import androidx.activity.EdgeToEdge;
import androidx.activity.result.ActivityResultLauncher;
import androidx.activity.result.contract.ActivityResultContracts;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.content.FileProvider;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;
```

```
import com.bumptech.glide.Glide;

import java.io.File;
import java.text.SimpleDateFormat;
import java.util.Date;
import java.util.Locale;

public class CameraActivity extends AppCompatActivity
{
    private ImageView ivFoto;
    private File fotoActual;
    private ActivityResultLauncher<Uri> tomarFotoLauncher;

    @Override
    protected void onCreate(Bundle savedInstanceState)
    {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable(this);
        setContentView(R.layout.activity_camera);
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.ma
in), (v, insets) ->
        {
            Insets systemBars =
insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top,
systemBars.right, systemBars.bottom);
            return insets;
        });

        ivFoto = findViewById(R.id.ivFoto);
    }
}
```

```
// Se registra el lanzador de la cámara (Activity Result
API).

// Debe registrarse aquí, en onCreate, no dentro de un
click.

tomarFotoLauncher = registerForActivityResult(
    new ActivityResultContracts.TakePicture(),
    éxito ->
    {
        if (Boolean.TRUE.equals(exito))
        {
            // La app de cámara ha guardado la foto
en fotoActual

            Glide.with(CameraActivity.this)
                .load(fotoActual)
                .into(ivFoto);
        }
    });

Button btnSacarFoto = findViewById(R.id.btnSacarFoto);
btnSacarFoto.setOnClickListener(new
View.OnClickListener()
{
    @Override
    public void onClick(View v)
    {
        abrirCamara();
    }
});

private void abrirCamara()
{
```

```
// Archivo destino en la carpeta propia de la app (no
requiere permisos)

File carpeta =
getExternalFilesDir(Environment.DIRECTORY_PICTURES);

String nombre = "foto_" + new
SimpleDateFormat("yyyyMMdd_HH:mm:ss",
Locale.getDefault()).format(new Date()) + ".jpg";

fotoActual = new File(carpeta, nombre);

// URI content:// segura mediante FileProvider

Uri uri = FileProvider.getUriForFile(this,
getPackageName() + ".fileprovider", fotoActual);

// Se lanza la app de cámara para que guarde la foto en
esa URI
tomarFotoLauncher.launch(uri);
}
}
```

Para capturar la foto se utiliza la Activity Result API, que es la forma actual de lanzar otra actividad y recibir su resultado (sustituye a los antiguos `startActivityForResult` y `onActivityResult`, ya descatalogados).

En `onCreate` se registra un lanzador (`ActivityResultLauncher`) mediante `registerForActivityResult`, al que se le pasan dos elementos: un contrato, en este caso `ActivityResultContracts.TakePicture()`, que describe la operación "hacer una foto"; y un callback, que se ejecutará cuando la cámara termine. El registro debe hacerse siempre en `onCreate` (no dentro del click del botón), ya que el sistema necesita conocer el lanzador antes de que la actividad se muestre. El contrato `TakePicture` devuelve un valor booleano que indica si la foto se realizó correctamente.

Al pulsar el botón se llama al método `abrirCamara`. En él se prepara primero el archivo de destino de la foto dentro de la carpeta privada de la aplicación, obtenida con `getExternalFilesDir(Environment.DIRECTORY_PICTURES)`. Esta ubicación es propia de la app y, por tanto, no requiere ningún permiso de almacenamiento. Al nombre del archivo se le añade una marca de tiempo para que cada foto sea única.

A continuación, como la app de cámara es una aplicación externa, no se le puede entregar directamente una ruta de archivo (por seguridad, Android no permite compartir rutas privadas entre apps). En su lugar se genera una URI de tipo `content://` mediante `FileProvider.getUriForFile`, que actúa de intermediario seguro y concede a la cámara permiso temporal para escribir en ese archivo concreto. El `FileProvider` se declaró

previamente en el `AndroidManifest.xml`, apuntando al archivo `file_paths.xml` que indica qué carpetas puede compartir.

Finalmente se lanza la cámara con `tomarFotoLauncher.launch(uri)`. La aplicación de cámara toma la fotografía y la guarda a resolución completa directamente en el archivo indicado. Cuando el proceso finaliza con éxito, el `callback` carga esa imagen en el `ImageView` mediante `Glide`, que se encarga de ajustarla al tamaño de la vista sin saturar la memoria (una foto de cámara a resolución completa es demasiado grande para cargarla tal cual).

Para implementar la galería la idea será listar la carpeta de fotos y mostrarlas en la cuadrícula con un adaptador. Se crea el archivo `res/layout/item_foto.xml`:

```
<?xml version="1.0" encoding="utf-8"?>

<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"

    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:gravity="center"
    android:padding="2dp">

    <ImageView
        android:id="@+id/ivItem"
        android:layout_width="120dp"
        android:layout_height="120dp"
        android:scaleType="centerCrop" />

</LinearLayout>
```

Se crea el adaptador que rellenará el `Grid View` en `adapter/FotosGridViewAdapter.java`:

```
package com.example.ejemplodialog.adapter;

import android.content.Context;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
```

```
import android.widget.BaseAdapter;
import android.widget.ImageView;

import com.bumptech.glide.Glide;
import com.example.ejemplodialog.R;

import java.io.File;
import java.util.List;

public class FotosGridViewAdapter extends BaseAdapter
{
    private final Context contexto;
    private final List<File> fotos;

    public FotosGridViewAdapter(Context contexto, List<File>
fotos)
    {
        this.contexto = contexto;
        this.fotos = fotos;
    }

    @Override
    public int getCount()
    {
        return fotos.size();
    }

    @Override
    public Object getItem(int position)
    {
        return fotos.get(position);
    }
}
```

```
@Override
public long getItemId(int position)
{
    return position;
}

@Override
public View getView(int position, View convertView,
    ViewGroup parent)
{
    LayoutInflater inflater =
    LayoutInflater.from(contexto);

    View fila = inflater.inflate(R.layout.item_foto,
    parent, false);

    ImageView imagen = fila.findViewById(R.id.ivItem);
    Glide.with(contexto)
        .load(fotos.get(position))
        .into(imagen);

    return fila;
}
}
```

Se agrega a CameraActivity.java la carga y refresco de la galería:

```
package com.example.ejemplodialog;

import android.net.Uri;
import android.os.Bundle;
import android.os.Environment;
import android.view.View;
```

```
import android.widget.Button;

import android.widget.GridView;

import android.widget.ImageView;


import androidx.activity.EdgeToEdge;
import androidx.activity.result.ActivityResultLauncher;
import
androidx.activity.result.contract.ActivityResultContracts;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.content.FileProvider;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

import com.bumptech.glide.Glide;
import com.example.ejemplodialog.adapter.FotosGridViewAdapter;

import java.io.File;
import java.text.SimpleDateFormat;
import java.util.ArrayList;
import java.util.Date;
import java.util.List;
import java.util.Locale;


public class CameraActivity extends AppCompatActivity
{
    private ImageView ivFoto;
    private GridView gvMiniaturas;
    private File fotoActual;
    private ActivityResultLauncher<Uri> tomarFotoLauncher;

    private final List<File> fotos = new ArrayList<>();
```

```
private FotosGridViewAdapter adapter;

@Override
protected void onCreate(Bundle savedInstanceState)
{
    super.onCreate(savedInstanceState);
    EdgeToEdge.enable(this);
    setContentView(R.layout.activity_camera);

    ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.ma
in), (v, insets) ->
    {
        Insets                systemBars                =
insets.getInsets(WindowInsetsCompat.Type.systemBars());
        v.setPadding(systemBars.left,                systemBars.top,
systemBars.right, systemBars.bottom);

        return insets;
    });

    ivFoto = findViewById(R.id.ivFoto);

    gvMiniaturas = findViewById(R.id.gvMiniaturas);
    adapter = new FotosGridViewAdapter(this, fotos);
    gvMiniaturas.setAdapter(adapter);

    // Se registra el lanzador de la cámara (Activity Result
API)
    tomarFotoLauncher = registerForActivityResult(
        new ActivityResultContracts.TakePicture(),
        exito ->
        {
            if (Boolean.TRUE.equals(exito))
            {

```

```

        Glide.with(CameraActivity.this)

            .load(fotoActual)

            .into(ivFoto);

        cargarImagenes();

    }

});

Button btnSacarFoto = findViewById(R.id.btnSacarFoto);

btnSacarFoto.setOnClickListener(new
View.OnClickListener()

{

    @Override

    public void onClick(View v)

    {

        abrirCamara();

    }

});

cargarImagenes();

}

private void abrirCamara()

{

    // Archivo destino en la carpeta propia de la app (no
    requiere permisos)

    File carpeta =
    getExternalFilesDir(Environment.DIRECTORY_PICTURES);

    String nombre = "foto_" + new
    SimpleDateFormat("yyyyMMdd_HH:mm:ss",
    Locale.getDefault()).format(new Date()) + ".jpg";

    fotoActual = new File(carpeta, nombre);

    // URI content:// segura mediante FileProvider

```

```

        Uri uri = FileProvider.getUriForFile(this,
        getPackageName() + ".fileprovider", fotoActual);

        tomarFotoLauncher.launch(uri);

    }

    private void cargarImagenes()
    {
        fotos.clear();

        File carpeta =
        getExternalFilesDir(Environment.DIRECTORY_PICTURES);

        if (carpeta != null)
        {
            File[] archivos = carpeta.listFiles();

            if (archivos != null)
            {
                for (File archivo : archivos)
                {
                    fotos.add(archivo);
                }
            }

            adapter.notifyDataSetChanged();
        }
    }
}

```

Podrá comprobarse que las fotos están en el Device Explorer en la ruta storage/emulated/0/Android/data/com.example.ejemplodialog/files/Pictures. Como las fotos se guardan en la carpeta privada de la app se borran al desinstalar la aplicación y no aparecen en la galería del sistema.

Para poder guardar las fotos en la galería del sistema habría que modificar el **AndroidManifest.xml** añadiendo la siguiente línea para solicitar permisos (solo en versiones viejas, de ahí el maxSdkVersion):

```
<uses-permission  
    android:name="android.permission.WRITE_EXTERNAL_STORAGE"  
    android:maxSdkVersion="28" />
```

Se modifica `FotosGridViewAdapter.java` para utilizar `Uri` en vez de `File`:

```
package com.example.ejemplodialog.adapter;  
  
import android.content.Context;  
import android.net.Uri;  
import android.view.LayoutInflater;  
import android.view.View;  
import android.view.ViewGroup;  
import android.widget.BaseAdapter;  
import android.widget.ImageView;  
  
import com.bumptech.glide.Glide;  
import com.example.ejemplodialog.R;  
  
import java.util.List;  
  
public class FotosGridViewAdapter extends BaseAdapter  
{  
    private final Context contexto;  
    private final List<Uri> fotos;  
  
    public FotosGridViewAdapter(Context contexto, List<Uri>  
fotos)  
    {  
        this.contexto = contexto;  
        this.fotos = fotos;  
    }  
}
```

```
@Override
public int getCount()
{
    return fotos.size();
}

@Override
public Object getItem(int position)
{
    return fotos.get(position);
}

@Override
public long getItemId(int position)
{
    return position;
}

@Override
public View getView(int position, View convertView,
    ViewGroup parent)
{
    LayoutInflater inflater =
    LayoutInflater.from(contexto);

    View fila = inflater.inflate(R.layout.item_foto,
    parent, false);

    ImageView imagen = fila.findViewById(R.id.ivItem);
    Glide.with(contexto)
        .load(fotos.get(position))
        .into(imagen);
}
```

```
        return fila;
    }
}
```

Se modifica CameraActivity.java:

```
package com.example.ejemplodialog;

import android.Manifest;
import android.content.ContentUris;
import android.content.ContentValues;
import android.content.pm.PackageManager;
import android.database.Cursor;
import android.net.Uri;
import android.os.Build;
import android.os.Bundle;
import android.os.Environment;
import android.provider.MediaStore;
import android.view.View;
import android.widget.Button;
import android.widget.GridView;
import android.widget.ImageView;
import android.widget.Toast;

import androidx.activity.EdgeToEdge;
import androidx.activity.result.ActivityResultLauncher;
import androidx.activity.result.contract.ActivityResultContracts;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.content.ContextCompat;
import androidx.core.content.FileProvider;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
```

```
import androidx.core.view.WindowInsetsCompat;

import com.bumptech.glide.Glide;
import com.example.ejemplodialog.adapter.FotosGridViewAdapter;

import java.io.File;
import java.io.FileInputStream;
import java.io.InputStream;
import java.io.OutputStream;
import java.text.SimpleDateFormat;
import java.util.ArrayList;
import java.util.Date;
import java.util.List;
import java.util.Locale;

public class CameraActivity extends AppCompatActivity
{
    private ImageView ivFoto;
    private GridView gvMiniaturas;
    private File fotoActual;
    private ActivityResultLauncher<Uri> tomarFotoLauncher;
    private ActivityResultLauncher<String> permisoLauncher;

    private final List<Uri> fotos = new ArrayList<>();
    private FotosGridViewAdapter adapter;

    @Override
    protected void onCreate(Bundle savedInstanceState)
    {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable(this);
    }
}
```

```

        setContentView(R.layout.activity_camera);

ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.ma
in), (v, insets) ->
    {
        Insets                systemBars                =
insets.getInsets(WindowInsetsCompat.Type.systemBars());

        v.setPadding(systemBars.left,                systemBars.top,
systemBars.right, systemBars.bottom);

        return insets;

    });

    ivFoto = findViewById(R.id.ivFoto);

    gvMiniaturas = findViewById(R.id.gvMiniaturas);
    adapter = new FotosGridViewAdapter(this, fotos);
    gvMiniaturas.setAdapter(adapter);

    // La cámara guarda la foto en un archivo propio
    (fiabile); luego se copia a la galería

    tomarFotoLauncher = registerForActivityResult(
        new ActivityResultContracts.TakePicture(),
        exito ->
        {
            if (Boolean.TRUE.equals(exito) &&
fotoActual != null)
            {
                guardarEnGaleria(fotoActual);

                Glide.with(CameraActivity.this).load(fotoActual).into(ivFoto);

                cargarImagenes();
            }
        });

```

```
// Permiso de escritura: solo se pide en Android 9 o
anterior

permisoLauncher = registerForActivityResult(
    new
    ActivityResultContracts.RequestPermission(),
    concedido ->
    {
        if (Boolean.TRUE.equals(concedido))
        {
            lanzarCamara();
        }
        else
        {
            Toast.makeText(this, "Sin permiso no se
            puede guardar en la galería", Toast.LENGTH_SHORT).show();
        }
    });

Button btnSacarFoto = findViewById(R.id.btnSacarFoto);
btnSacarFoto.setOnClickListener(new
View.OnClickListener()
{
    @Override
    public void onClick(View v)
    {
        abrirCamara();
    }
});

cargarImagenes();
}

private void abrirCamara()
```

```
{
    // En Android 9 o anterior hace falta permiso de
    escritura; en Android 10+ no

    if (Build.VERSION.SDK_INT < Build.VERSION_CODES.Q &&
        ContextCompat.checkSelfPermission(this,
        Manifest.permission.WRITE_EXTERNAL_STORAGE) !=
        PackageManager.PERMISSION_GRANTED)
    {

        permisoLauncher.launch(Manifest.permission.WRITE_EXTERNAL_STOR
        AGE);

        return;
    }

    lanzarCamara();
}

private void lanzarCamara()
{
    // 1) Se captura a un archivo propio de la app mediante
    FileProvider (fiable)

    File carpeta =
    getExternalFilesDir(Environment.DIRECTORY_PICTURES);

    String nombre = "foto_" + new
    SimpleDateFormat("yyyyMMdd HHmmss",
    Locale.getDefault()).format(new Date()) + ".jpg";

    fotoActual = new File(carpeta, nombre);

    Uri uri = FileProvider.getUriForFile(this,
    getPackageName() + ".fileprovider", fotoActual);

    tomarFotoLauncher.launch(uri);
}

// 2) Se copia el archivo capturado al MediaStore para que
    aparezca en la galería del sistema

    private void guardarEnGaleria(File archivo)
```

```
{
    ContentValues valores = new ContentValues();

    valores.put(MediaStore.Images.Media.DISPLAY_NAME,
archivo.getName());

    valores.put(MediaStore.Images.Media.MIME_TYPE,
"image/jpeg");

    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.Q)
    {
        valores.put(MediaStore.Images.Media.RELATIVE_PATH,
Environment.DIRECTORY_PICTURES + "/EjemploDialog");

        valores.put(MediaStore.Images.Media.IS_PENDING,
1);
    }

    Uri destino =
getContentResolver().insert(MediaStore.Images.Media.EXTERNAL_C
ONTENT_URI, valores);

    if (destino == null)
    {
        return;
    }

    // Se copian los bytes del archivo a la entrada del
MediaStore

    try (InputStream in = new FileInputStream(archivo);

        OutputStream out =
getContentResolver().openOutputStream(destino))
    {
        byte[] buffer = new byte[8192];

        int leidos;

        while ((leidos = in.read(buffer)) > 0)
        {
            out.write(buffer, 0, leidos);
        }
    }
}
```

```

    }

    catch (Exception e)
    {
        e.printStackTrace();
    }

    // A partir de Android 10, se marca como "ya no
    pendiente" para hacerla visible

    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.Q)
    {
        ContentValues v = new ContentValues();
        v.put(MediaStore.Images.Media.IS_PENDING, 0);
        getContentResolver().update(destino, v, null,
        null);
    }
}

private void cargarImagenes()
{
    fotos.clear();

    Uri coleccion =
    MediaStore.Images.Media.EXTERNAL_CONTENT_URI;

    String[] proyeccion = { MediaStore.Images.Media._ID };
    String seleccion = MediaStore.Images.Media.DISPLAY_NAME
    + " LIKE ?";
    String[] args = { "foto_" };
    String orden = MediaStore.Images.Media.DATE_ADDED + "
    DESC";

    try (Cursor cursor =
    getContentResolver().query(coleccion, proyeccion, seleccion,
    args, orden))
    {

```

```

        if (cursor != null)
        {
            int idCol =
            cursor.getColumnIndexOrThrow(MediaStore.Images.Media._ID);

            while (cursor.moveToNext())
            {
                long id = cursor.getLong(idCol);

                fotos.add(ContentUris.withAppendedId(coleccion, id));
            }
        }
    }

    adapter.notifyDataSetChanged();
}
}

```

En esta ampliación, además de capturar la foto en la carpeta propia de la aplicación, se copia al MediaStore (el almacén compartido de imágenes del sistema), de modo que aparezca en la galería, pueda compartirse con otras apps y no se borre al desinstalar.

La captura sigue haciéndose con la Activity Result API, pero ahora en dos fases. Primero, el método lanzarCamara crea un archivo propio en la carpeta de la app (con getExternalFilesDir) y entrega su Uri de FileProvider a la cámara, que escribe la foto en él; es la parte fiable, la misma que ya funcionaba en la versión anterior. Segundo, cuando la captura termina con éxito, el método guardarEnGaleria copia ese archivo al MediaStore: crea una entrada con getContentResolver().insert(...) —indicando el nombre (DISPLAY\_NAME), el tipo (MIME\_TYPE) y, en Android 10 o superior, la carpeta pública Pictures/EjemploDialog— y vuelca en ella los bytes del archivo.

En Android 10 y superiores se emplea la marca IS\_PENDING: al crear la entrada se pone a 1 (la imagen se está escribiendo y no debe mostrarse "a medias") y, una vez copiada, se pone a 0 con getContentResolver().update(...), momento en el que la foto se hace visible para el resto del sistema.

En cuanto a los permisos, hay una diferencia según la versión de Android. Desde Android 10 (API 29), añadir imágenes propias al MediaStore no requiere ningún permiso, gracias al scoped storage. En Android 9 o anterior sí es necesario WRITE\_EXTERNAL\_STORAGE, por lo que el método abrirCamara comprueba la versión del dispositivo y solo en las antiguas lo solicita en tiempo de ejecución (contrato RequestPermission). En

el `AndroidManifest.xml` ese permiso se declara con `android:maxSdkVersion="28"` para que solo se tenga en cuenta donde de verdad hace falta.

Por último, la galería de la propia aplicación consulta el `MediaStore` con `getContentResolver().query(...)`, pidiendo las imágenes cuyo nombre empieza por `foto_` y ordenándolas de más reciente a más antigua. Por cada resultado construye su `Uri` con `ContentUris.withAppendedId(...)` y la añade a la lista que alimenta el adaptador. Como `Glide` carga imágenes tanto desde un `File` como desde una `Uri`, el adaptador apenas cambia: solo pasa a trabajar con `Uri`.

Centro de Estudios

**almi**  
IKASTETXEA

SDTAG